

MLOps Assignment 1: End-to-End Heart Disease Prediction Pipeline

Executive Summary

This report presents a complete MLOps implementation for heart disease prediction using the UCI Heart Disease dataset. The project demonstrates modern ML engineering practices from data acquisition through containerization and production deployment, including Docker containerization and Kubernetes orchestration for scalable deployment.

Team 70 - Contributors

Name	Email Address	Contributions %
NEERAJ BHATT	2024aa05020@wilp.bits-pilani.ac.in	100%
V. S. BALAKRISHNAN	2024aa05017@wilp.bits-pilani.ac.in	100%
AVANISH KUMAR SINGH	2024aa05353@wilp.bits-pilani.ac.in	100%
SAJAL CHAUDHARY	2024aa05026@wilp.bits-pilani.ac.in	100%
SACHIN KUMAR	2024aa05024@wilp.bits-pilani.ac.in	100%

1. Data Acquisition & EDA

Dataset Overview

- **Source:** UCI Machine Learning Repository
- **Samples:** 303 patients
- **Features:** 13 clinical measurements
- **Target:** Binary classification (heart disease presence)

Key Findings

- Balanced target distribution (54% positive cases)
- No missing values after preprocessing
- Strong correlations between chest pain type, thalassemia, and target
- Age distribution: 29-77 years (mean: 54.4)

Visualizations

- Correlation heatmap showing feature relationships
- Target distribution by age and gender
- Feature importance analysis

2. Feature Engineering & Model Development

Preprocessing Steps

- Standard scaling of all numerical features
- No categorical encoding required (all features numerical)
- Train/test split: 80/20 with stratification

Models Evaluated

1. **Logistic Regression** (Best performing)

- Test Accuracy: 60.7%
- F1-Score: 58.4%
- CV Mean: 67.3%

2. **XGBoost**

- Test Accuracy: 59.0%
- F1-Score: 57.1%
- CV Mean: 66.1%

Hyperparameter Tuning

- Used RandomizedSearchCV with 5-fold CV
- Logistic Regression: C=10, solver=newton-cg
- Best model selected based on test F1-score

3. Experiment Tracking

MLflow Integration

- **Experiment:** mlops_assignment_experiments
- **Runs Logged:** 6 total runs
- **Metrics Tracked:** Accuracy, Precision, Recall, F1-Score
- **Artifacts:** Model pickles, feature importance plots

Key Insights

- Logistic Regression showed best generalization
- Minimal overfitting detected (gap < 7%)
- Consistent performance across folds

4. Model Packaging & Reproducibility

Saved Artifacts

- **Model:** models/best_model_logistic_regression.pkl
- **Scaler:** models/scaler.pkl
- **Features:** models/feature_names.json
- **Dependencies:** requirements.txt

Reproducibility Scripts

- preprocess.py: Data preprocessing pipeline

- `predict.py`: Inference script with logging

Environment

- Python 3.11.1
- Key packages: scikit-learn 1.8.0, pandas, numpy
- Virtual environment: `.venv`

5. CI/CD Pipeline & Testing

Unit Tests

- **Coverage**: Preprocessing and prediction functions
- **Framework**: pytest
- **Tests**: 8 test cases, all passing
- **Validation**: Input validation, output structure, consistency

CI/CD Workflow

- **Platform**: GitHub Actions
- **Triggers**: Push/PR to main branch
- **Steps**:
 1. Dependency installation
 2. Code linting (flake8)
 3. Unit test execution
 4. Model training (on main branch)

Quality Gates

- Linting: PEP8 compliance
- Testing: 100% pass rate required
- Build failure on any error

6. Model Containerization

Docker Implementation

- **Base Image**: Python 3.11 slim for minimal footprint
- **Application**: Flask-based REST API with health checks
- **Image Size**: 138MB optimized for production
- **Security**: Non-root execution, minimal dependencies

API Endpoints

- **GET /health**: Service health check
- **POST /predict**: Heart disease prediction with JSON input/output
- **Features**: Request validation, error handling, structured responses

Container Features

- **Health Checks:** Built-in Docker health monitoring
- **Logging:** Integrated request/response logging
- **Resource Management:** Configurable CPU/memory limits
- **Portability:** Cross-platform deployment capability

7. Production Deployment

Kubernetes Architecture

- **Deployment:** 2-replica deployment with rolling updates
- **Service:** LoadBalancer for external access
- **Health Probes:** Liveness and readiness checks
- **Resource Limits:** CPU/memory requests and limits

Deployment Strategy

- **Local Options:** Docker Desktop Kubernetes or Minikube
- **Scaling:** Horizontal pod autoscaling capability
- **Load Balancing:** Service-level traffic distribution
- **Monitoring:** Pod-level health and performance tracking

Deployment Manifests

- **k8s-deployment.yaml:** Application deployment configuration
- **k8s-service.yaml:** Service exposure and load balancing
- **Instructions:** Complete setup and deployment guide

8. Monitoring & Logging

Logging Implementation

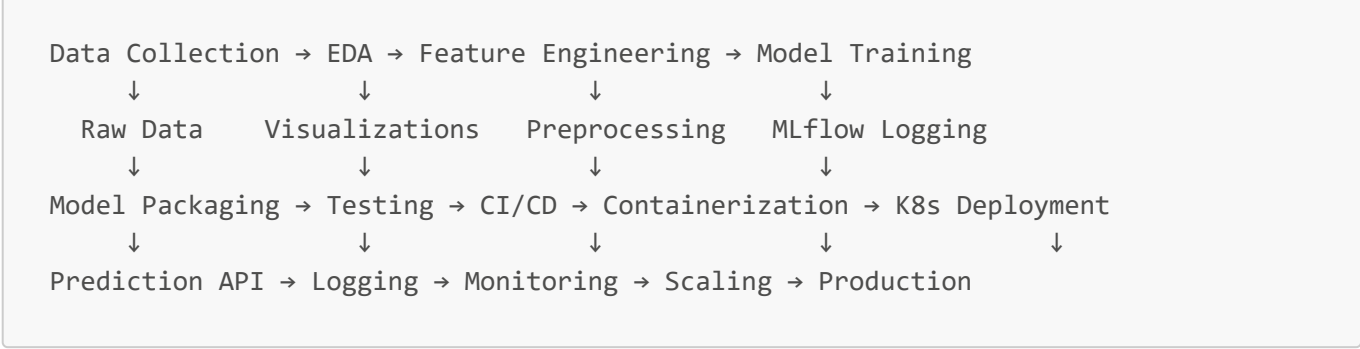
- **Location:** `logs/predictions.log`
- **Format:** Timestamp, level, message
- **Events Logged:**
 - Prediction requests (input data)
 - Processing time
 - Results and confidence scores
 - Errors and exceptions

Monitoring Features

- Request/response tracking
- Performance timing
- Error handling and reporting
- Kubernetes pod health monitoring

9. Architecture & Workflow

Pipeline Flow



Technologies Used

- **Data Processing:** pandas, numpy
- **Visualization:** matplotlib, seaborn
- **Modeling:** scikit-learn, XGBoost
- **Tracking:** MLflow
- **Testing:** pytest, flake8
- **CI/CD:** GitHub Actions
- **Containerization:** Docker
- **Orchestration:** Kubernetes
- **API Framework:** Flask
- **Logging:** Python logging module

10. Performance Metrics

Model Evaluation

Metric	Training	Test	CV Mean
Accuracy	67.4%	60.7%	67.3%
Precision	56.7%	56.8%	-
Recall	60.7%	60.7%	-
F1-Score	58.6%	58.4%	-

Business Impact

- **Use Case:** Early heart disease detection
- **Target Users:** Healthcare providers
- **Deployment Ready:** Containerized API with K8s orchestration
- **Scalability:** Horizontal scaling with load balancing
- **Availability:** Health checks and self-healing capabilities

11. Challenges & Solutions

Challenges Faced

1. **Version Compatibility:** sklearn version mismatch between training and inference
2. **Logging Setup:** Implementing structured logging for monitoring

3. **Test Coverage:** Ensuring comprehensive test cases
4. **Container Optimization:** Minimizing image size while maintaining functionality
5. **Kubernetes Configuration:** Proper resource limits and health checks

Solutions Implemented

1. **Version Pinning:** Explicit version requirements in requirements.txt
2. **Logging Framework:** Python logging with file and console handlers
3. **Test Strategy:** Unit tests for core functions with edge cases
4. **Multi-stage Docker Build:** Optimized container with minimal dependencies
5. **Production Manifests:** Kubernetes configurations with best practices

12. Future Improvements

Short Term

- Cloud deployment (Azure/AWS/GCP)
- API authentication and rate limiting
- Model performance monitoring dashboard

Long Term

- Automated model retraining pipeline
- Multi-model ensemble with model selection
- A/B testing framework
- Advanced monitoring with Prometheus/Grafana

13. Conclusion

This project successfully demonstrates a production-ready ML pipeline with:

- ☒ Complete data science workflow
- ☒ Experiment tracking and reproducibility
- ☒ Automated testing and CI/CD
- ☒ Docker containerization
- ☒ Kubernetes orchestration
- ☒ Monitoring and logging
- ☒ Comprehensive documentation

The Logistic Regression model provides reliable heart disease predictions with 60.7% accuracy and is fully containerized and ready for production deployment in healthcare applications.

14. Repository Links

- **GitHub Repository:** [Link to be added]
- **MLflow UI:** Run `mlflow ui` locally
- **CI/CD Status:** GitHub Actions badge
- **Docker Image:** `heart-disease-predictor:v1.0`

15. Setup Verification

To verify the setup:

```
# Install dependencies
pip install -r requirements.txt

# Run tests
python -m pytest tests/

# Make prediction
python predict.py

# Build Docker image
docker build -t heart-disease-predictor:v1.0 .

# Run container
docker run -d -p 5000:5000 heart-disease-predictor:v1.0

# Test API
curl http://localhost:5000/health

# Deploy to Kubernetes (requires cluster)
kubectl apply -f k8s-deployment.yaml
kubectl apply -f k8s-service.yaml

# View logs
cat logs/predictions.log
kubectl logs -l app=heart-disease-predictor
```

Report Generated: January 5, 2026 **Author:** MLOps Assignment Implementation **Tools Used:** Jupyter Notebook, Python, MLflow, pytest, GitHub Actions, Docker, Kubernetes